	account.
Condition	The signal and the test, such as failed pipeline runs greater than 0 within 5 minutes.
Severity	From Sev 0 (critical) to Sev 4 (verbose). It helps decide how fast someone must react
Action group	A reusable list of who is told and what runs: email, SMS, push message, voice call, webhook, Azure Function, or Logic App.
Alert type	**Fires when**
Metric alert	A metric crosses a threshold, such as failed pipeline runs above 0 or storage availability below a set level.
Log search alert	A KQL query returns rows or a count above a limit, such as more than five failed requests in ten minutes.
Activity log alert	A management action happens, such as a resource being deleted, or Azure reports a service health event.

After an alert fires. It appears under Alerts in Azure Monitor with its severity. A metric alert is marked resolved when the condition clears, and a person can set the alert to acknowledged or closed to show who is handling it.

SESSION 5 — AZURE II: DATABRICKS CONNECTIVITY AND CI/CD

- **Topic:** Azure II — Databricks Connectivity and CI/CD
- **Date:** 28th October 2026

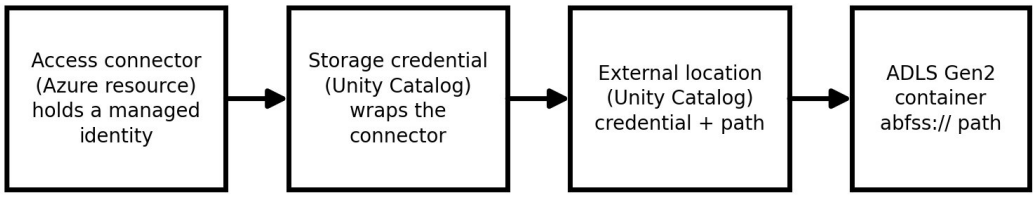
15. Databricks Access Connector and Unity Catalog Storage Credentials

In simple words. Databricks needs a safe way to open the storage account without anyone pasting a key into a notebook. Unity Catalog, the governance layer of Databricks, does this with a managed identity (section 6). The identity lives in an Azure resource called the Access Connector for Azure Databricks, and Unity Catalog wraps it in an object called a storage credential.

Piece	**Simple meaning**
Access connector	An Azure resource (Microsoft.Databricks/accessConnectors) that holds a managed identity. It is created in the Azure portal, in

	the same region as the workspace, and it can use a system-assigned or a user-assigned identity.
Role assignment	The Azure RBAC role given to that identity on the storage account, such as Storage Blob Data Contributor to read and write, or Storage Blob Data Reader to only read (sections 7 to 9).
Storage credential	A Unity Catalog object that stores the access connector's resource ID. It is the permission to sign in to storage, and it holds no password or key.

How Unity Catalog reaches the storage account



The identity also needs Storage Blob Data Contributor on the storage account

Diagram 12: The connector's identity is trusted by Azure, wrapped by Unity Catalog, and tied to a path.

Step	**What to do**
1. Have the storage ready	Use the ADLS Gen2 account with the hierarchical namespace on, such as stdatalakedev001.
2. Create the connector	In the Azure portal create an Access Connector for Azure Databricks in the resource group, and copy its resource ID.
3. Give it a role	On the storage account, give the connector's identity Storage Blob Data Contributor, or give it only on the one container it needs.
4. Create the credential	In Databricks open Catalog, then External data, then Credentials, and create a storage credential with the connector's resource ID. For a user-assigned identity, add that identity's resource ID as well.
5. Set who can use it	Grant privileges on the credential only to the people who manage external locations, as the next section

	shows.

16. Unity Catalog External Locations

In simple words. A storage credential says how to sign in, and an external location says where. An external location joins one storage path, such as **abfss://bronze@stdatalakedev001.dfs.core.windows.net/** to one storage credential. Once it exists, Unity Catalog decides who may read or write files under that path.

Privilege	What it allows
READ FILES	Read the files under the location, for example with spark.read or read_files.
WRITE FILES	Write, change, and delete files under the location.
CREATE EXTERNAL TABLE	Create a table whose files stay in the location.
CREATE EXTERNAL VOLUME	Create a volume for files that are not tables, such as images or PDFs.
CREATE MANAGED STORAGE	Use the location as managed storage for a catalog or a schema.

```
-- Create the external location (needs CREATE EXTERNAL LOCATION)
CREATE EXTERNAL LOCATION IF NOT EXISTS ext_bronze
URL 'abfss://bronze@stdatalakedev001.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL cred_datalake_dev)
COMMENT 'Raw files for the TriCloud data lake';

-- Let a group read files from it
GRANT READ FILES ON EXTERNAL LOCATION ext_bronze TO `data-engineers`;

-- External table: the files stay in this path (it already holds a Delta table)
CREATE TABLE main.bronze.sales_raw
LOCATION 'abfss://bronze@stdatalakedev001.dfs.core.windows.net/sales/';
```

Point	Managed table vs. external table
Where the files live	Managed: in the managed storage of Unity Catalog, set for the metastore, a catalog, or a schema. External: in your own path, covered by an external location.
When the table is	Managed: the data files are removed with the table.

dropped	External: only the table definition is removed, and the files stay.
Best for	Managed: new tables, where Unity Catalog can take care of the files. External: data that other tools also read, or data that must stay where it is.

- **Paths must not overlap.** Do not create an external location inside the path of another one. Unity Catalog blocks overlapping paths so that one path never has two owners.
- **Test the connection.** The Test connection button on an external location checks that the credential can really reach the path, which finds a missing role assignment early.
- **Reading a path directly.** A user can read abfss:// files directly only if they hold READ FILES on the external location that covers the path. Without it, the request is denied even if the path exists.
- **Link to Data Factory.** Data Factory writes raw files into the bronze container (section 12), and Databricks reads them through the external location, so each side uses its own identity and neither needs a key.

17. Azure DevOps CI/CD Fundamentals

In simple words. Azure DevOps is Microsoft's set of tools for building software as a team: Boards for tasks, Repos for Git code, Pipelines for automation, Test Plans for testing, and Artifacts for packages. CI/CD means that every change to the code is checked and delivered by a repeatable automatic process and not by hand.

Term	Simple meaning
Continuous integration (CI)	Every push or pull request is built and tested automatically, so a broken change is found within minutes.
Continuous delivery (CD)	A change that passed CI is released to each environment by the same steps every time. Production can wait for a manual approval, or it can deploy automatically.
Pipeline	A YAML file in the repository, usually azure-pipelines.yml, that describes the automatic steps. It is stored and reviewed like any other code.
Trigger	What starts the pipeline: a push to a branch, a pull request, or a schedule.
Stage, job, step	A pipeline has stages such as Build and Deploy. A stage holds jobs, and a job runs steps, such as a script or a task.
Agent	The machine that runs a job. A Microsoft-hosted agent is a fresh virtual machine, and a self-hosted agent runs on your own machine.
Artifact	The output of the build, such as the packaged notebooks, saved once and deployed unchanged to every environment.
Environment	A named target such as dev or prod, where approvals and checks can be added before a deployment is allowed to run.

Service connection	The saved sign-in that lets a pipeline reach Azure. The workload identity federation option is preferred, because it stores no secret that can expire or leak.
Variable group	Shared settings for pipelines. It can read its secrets from Azure Key Vault (section 10).

A simple CI/CD flow in Azure DevOps

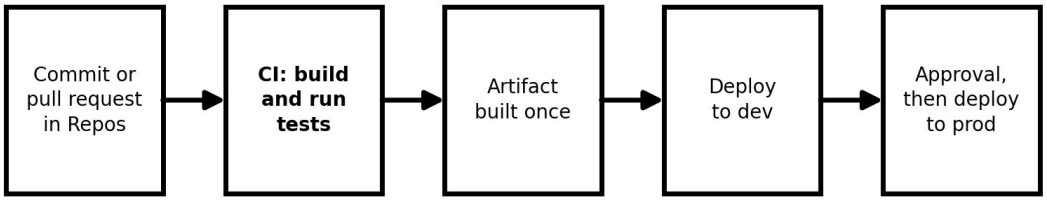


Diagram 13: The code is built and tested once, and the same artifact moves from dev to prod.

A working flow. A person creates a feature branch and opens a pull request into main. A branch policy runs the build and tests on the pull request, and the merge is allowed only when they pass. The merge starts the pipeline again, which builds the artifact, deploys it to dev, and then waits for an approval before it deploys to prod.